MyList App Development Report

A SwiftUI Task-Management Application

Student: _____

Professor: _____

Course: _____

Date: _____

Introduction

For this project, I created **MyList**, a task-management application using Swift, SwiftUI, and PencilKit. The purpose of the app is to let users organize tasks into groups, track their progress, and create tasks by either typing or drawing.

While developing this project, I learned how to design an adaptive SwiftUI interface, manage state between views, save user data, support dark mode, and integrate a UIKit framework into a SwiftUI application.

Programming Language and Frameworks

The main programming language used for the project was **Swift**. I used several Apple frameworks:

- **SwiftUI** for the application interface and navigation.
- **Foundation** for data storage, JSON encoding, UUID values, and UserDefaults.
- **PencilKit** for drawing tasks with a finger or Apple Pencil.
- **UIKit integration** to display PKCanvasView inside SwiftUI.

I learned how Swift's strong type system, structures, optionals, enumerations, computed properties, and closures can be combined to create an interactive application.

Data Models

I created two main data models: `TaskItem` and `TaskGroup`.

A `TaskItem` stores a unique identifier, task title, completion status, and optional PencilKit drawing data. A `TaskGroup` stores a unique identifier, group title, SF Symbol name, and an array of tasks.

Both models conform to `Identifiable`, `Hashable`, and `Codable`. Identifiable allows the models to work correctly inside SwiftUI lists. Hashable supports navigation and selection. Codable allows the models to be converted into data and saved.

Saving Data with UserDefaults

I learned how to preserve the user's task groups by combining `UserDefaults` with `JSONEncoder` and `JSONDecoder`.

When the app moves into the background, the task-group array is encoded:

```
let encodedData = try? JSONEncoder().encode(taskGroups)
```

The encoded data is then stored in UserDefaults:

```
UserDefaults.standard.set(encodedData, forKey: saveKey)
```

When the app opens, it retrieves the saved data and decodes it back into an array of `TaskGroup` objects. If no saved data exists, the app loads sample data instead. This taught me that UserDefaults can store simple values directly, but custom Swift models must first be converted into Data.

Application Lifecycle

I used the SwiftUI scene-phase environment value:

```
@Environment(\.scenePhase) private var scenePhase
```

This allows the app to detect when it is active, inactive, or running in the background. When the app enters the background, it calls the data-saving function. This helped me understand that saving data should happen at appropriate points in the application lifecycle instead of relying only on button presses.

Dark Mode

I implemented a light and dark appearance toggle using a toolbar button. The button changes between sun and moon SF Symbols so the user can understand what appearance will be selected.

The setting uses `@AppStorage`, which is SwiftUI's convenient property wrapper for a value stored in UserDefaults:

```
@AppStorage("isDarkMode") private var isDarkMode = false
```

The selected appearance is applied using:

```
.preferredColorScheme(isDarkMode ? .dark : .light)
```

This feature taught me the difference between reading system settings through `@Environment`, storing preferences with UserDefaults or AppStorage, and applying a preferred color scheme to a SwiftUI view hierarchy.

Split-Screen Navigation

The primary interface uses `NavigationSplitView`. It separates the app into a sidebar and a detail area. The sidebar displays all task groups, while the detail area displays the tasks belonging to the selected group. If the user has not selected a group, the app presents a `ContentUnavailableView`.

This design works especially well on iPad because both columns can appear simultaneously. On smaller devices, SwiftUI automatically adapts the navigation interface to the available space. I also used `NavigationSplitViewVisibility` to manage the visibility of the split-view columns.

Responsive Layout and Resizing

I learned how to make the application adjust to different device sizes. The detail view reads the horizontal size class:

```
@Environment(\.horizontalSizeClass) var sizeClass
```

When the app has a regular-width layout, such as on many iPad configurations, it displays additional task statistics. On compact screens, that section is hidden to preserve space.

The icon selector uses an adaptive LazyVGrid:

```
GridItem(.adaptive(minimum: 40))
```

This allows the number of icon columns to change automatically as the window or device width changes. These features helped me understand responsive layouts and how SwiftUI can adapt an interface without requiring separate screens for every device size.

Creating Task Groups

Users can create custom task groups from a modal sheet. The new-group screen includes a text field for the group name, an adaptive grid of SF Symbols, visual feedback for the selected symbol, Cancel and Save toolbar buttons, and a disabled Save button when the group name is empty.

The completed group is returned to the main view through a closure. This taught me how child views can communicate results back to their parent views.

Creating Tasks

The new-task screen provides two input modes: **Type**, for entering a task with a text field, and **Draw**, for creating a handwritten or sketched task. I represented these options with an `InputMode` enumeration and displayed them using a segmented picker.

A computed property determines whether the task is valid. For typed tasks, the title cannot be empty. For drawn tasks, the canvas must contain at least one stroke. The Save button remains disabled until the selected input is valid.

Drawing with PencilKit

One of the most important features I implemented was PencilKit drawing. Because `PKCanvasView` is a UIKit view, I created a `UIViewRepresentable` wrapper named `CanvasView`. This allowed me to display PencilKit content inside the SwiftUI interface.

The wrapper creates and configures PKCanvasView, permits Apple Pencil and finger input, displays PKToolPicker, monitors drawing changes through a coordinator, and removes tool-picker observers when the view is dismissed.

The drawing screen also includes Clear, Undo, and Redo controls with keyboard shortcuts. When a drawing task is saved, the PencilKit drawing is converted into Data:

```
canvasView.drawing.dataRepresentation()
```

The detail view later reconstructs the drawing using PKDrawing and displays it as an image. This taught me how SwiftUI and UIKit can work together when a required component is not a native SwiftUI view.

State Management and Data Flow

I used several SwiftUI state-management tools:

- `@State` for temporary values owned by a view.
- `@Binding` for editing data owned by a parent view.
- `@Environment` for system-provided values such as dismissal, scene phase, and size class.
- `@AppStorage` for a persistent appearance preference.

For example, the detail view receives a binding to a TaskGroup. This allows it to edit tasks directly while keeping the main task-group array synchronized. This helped me understand that a view should own local state, while shared editable data should be passed using bindings.

Task Completion and Deletion

Users can mark a task as complete by tapping its circular icon. The app uses animation when changing the completion state. Completed typed tasks receive a checkmark icon, a strikethrough effect, and a gray foreground color. Completed drawing tasks become partially transparent.

Users can also delete tasks with the standard SwiftUI list deletion gesture. These features taught me how to combine bindings, conditional styling, animations, and list actions.

Progress Statistics

I created `GroupStatsView` to calculate and display task progress. It uses computed properties to determine how many tasks are complete and the completion percentage.

The progress is displayed using two layered circles. The foreground circle is trimmed based on the completion percentage and rotated so progress begins at the top. This taught me how to derive interface values from existing model data instead of storing duplicate state.

Accessibility and System Design

I used SF Symbols throughout the application so icons remain clear and consistent with Apple platforms. The dark-mode button also includes an accessibility label that changes according to its current action.

The interface uses semantic system colors such as primary, gray, systemBackground, and secondarySystemBackground. These colors adapt more effectively to appearance changes than fixed colors.

Challenges and Problem Solving

Some of the main challenges included saving nested custom models in UserDefaults, passing editable data between multiple views, making navigation work on compact and regular screens, integrating a UIKit drawing view with SwiftUI, converting PencilKit drawings to data and restoring them, making dark mode affect the correct view hierarchy, validating two different task-input modes, and keeping the interface responsive when the window size changes.

Solving these problems improved my understanding of debugging, compiler diagnostics, data flow, and breaking larger features into smaller components.

Conclusion

Developing MyList gave me practical experience with Swift and SwiftUI beyond building a basic list. I learned how to create reusable models and views, persist data, respond to application lifecycle changes, build adaptive split-screen navigation, support appearance changes, and integrate PencilKit drawing.

The project also helped me understand how SwiftUI's declarative approach works: the interface updates automatically when its underlying state changes. Overall, MyList demonstrates task organization, persistent storage, responsive design, user preferences, progress visualization, and both typed and handwritten input.